# Backend Development Assignment — حلويات الحلاج

**نوع المشروع:** RESTful API لتطبيق طلب الحلويات

**موعد التسليم النهائي:** 22/10/2026

**التقنيات المطلوبة:** **JavaScript**, **Node.js**, **Express.js**, **MongoDB**, **Mongoose**, **Morgan**, **jsonwebtoken (JWT)**, **Nodemon**

---

## 1. معلومات التكليف

- **اسم المشروع:** نظام الطلبات الإلكترونية — حلويات الحلاج.
- **نوع المشروع:** **RESTful API** لتطبيق طلب الحلويات.
- **موعد التسليم النهائي:** 22/10/2026.
- **التقنيات المطلوبة:** **JavaScript (JS)**, **Node.js**, **Express.js**, **MongoDB**, **Mongoose**, **Morgan**, **jsonwebtoken (JWT)**, **Nodemon**.

# 2. قصة المشروع والمشكلة

حلويات الحلاج محل حلويات شهير في المحافظة، ويرغب صاحب المحل في تطوير طريقة التواصل مع الزبائن وتسريع عملية الطلب.

تتمثل المشكلة الحالية في اضطرار الزبائن إلى زيارة المحل لإتمام طلباتهم، مما يستهلك وقتهم ويزيد أعباء التنقل، كما يحد من قدرة المتجر على استقبال الطلبات وإدارتها بكفاءة.

تواصل صاحب المحل معك بصفتك مطور برمجيات، وطلب منك تصميم Backend لتطبيق موبايل يتيح للزبائن استعراض المنتجات وإرسال طلباتهم من المنزل، مع توفير نظام إداري يساعد صاحب المحل والموظفين على إدارة المنتجات والطلبات والمستخدمين.

مهم: المطلوب في هذا التكليف هو تطوير الـ **Backend** فقط، ولا يشمل تصميم تطبيق الموبايل أو واجهات **Frontend**.

# 3. أنواع المستخدمين والصلاحيات

يجب أن يدعم النظام ثلاثة أنواع من الحسابات:

## أ. صاحب المحل (Admin)

يمتلك الصلاحيات الإدارية الكاملة، بما في ذلك:

- إنشاء حسابات الموظفين.
- تعديل بيانات الموظفين وكلمات مرورهم.
- حظر الموظفين أو حذف حساباتهم.
- الاطلاع على بيانات المستخدمين وإدارتها.
- حظر أي مستخدم مع تسجيل سبب الحظر.
- إلغاء الحظر عند الحاجة.
- الاطلاع على المنتجات والأقسام والطلبات.

- الوصول إلى إحصائيات عامة عن المبيعات والطلبات والمنتجات والمستخدمين.

## ب. الموظف (Employee)

يكون مسؤولًا عن العمليات اليومية، وتشمل صلاحياته:

- إنشاء أقسام الحلويات وتعديلها وحذفها.
- إضافة المنتجات وتعديلها وحذفها.
- تحديد وصف كل منتج والأسعار والكميات المتاحة للطلب.
- الاطلاع على طلبات الزبائن وإدارتها.
- الموافقة على الطلبات المقبولة وتحديث حالتها.
- تحديث حالة الطلب أثناء التجهيز والتوصيل.

يجب ألا يتمكن الموظف من الوصول إلى الصلاحيات الإدارية الخاصة بصاحب المحل.

## ج. الزبون (Customer)

يجب أن يمتلك كل زبون حسابًا خاصًا، ولا يُسمح بتنفيذ الطلبات كضيف. يستطيع الزبون:

- إنشاء حساب باستخدام رقم الهاتف وكلمة المرور.
- تسجيل الدخول والخروج.
- استعراض أقسام الحلويات والمنتجات المتاحة.
- الاطلاع على وصف المنتجات وأسعارها وخيارات الكمية.
- إنشاء طلب يحتوي على منتج واحد أو عدة منتجات.
- تحديد عنوان التوصيل كنص.
- استعراض عناوين التوصيل السابقة واختيار أحدها.
- الاطلاع على طلباته السابقة والحالية وحالة كل طلب.
- الاطلاع على تفاصيل الطلب.

يجب منع الزبون المحظور من استخدام الخدمات المحمية، مع حفظ سبب الحظر وإمكانية إلغاء الحظر من قبل صاحب المحل.

# 4. إدارة أقسام الحلويات والمنتجات

يجب أن يسمح النظام بإنشاء أقسام مختلفة، مثل:

- الحلويات الشرقية.
- الحلويات الغربية.
- المعجنات.
- الشوكولاتة.
- أقسام أخرى يمكن إضافتها مستقبلًا.

يحتوي كل قسم على اسم ووصف اختياري وحالة تفعيل.

ويرتبط كل منتج بأحد الأقسام، ويحتوي على الأقل على:

- اسم المنتج.
- وصف المنتج.
- السعر.
- القسم التابع له.
- خيارات الكمية المتاحة للطلب، مثل نصف كيلو أو كيلو أو أكثر.
- حالة التوفر أو التفعيل.
- تاريخ الإنشاء والتحديث.

لا يُسمح باستخدام الصور في هذا المشروع. يجب تنفيذ إدارة المنتجات بالاعتماد على البيانات النصية فقط.

ينبغي مراعاة أن خيارات الكمية والأسعار يجب أن تكون واضحة في تصميم قاعدة البيانات، وأن يتمكن الموظف من تحديثها بسهولة.

# 5. نظام الطلبات

يجب أن يتمكن الزبون من إنشاء طلب يحتوي على عدة منتجات، مع تحديد الكمية المطلوبة لكل منتج.

يتضمن الطلب، على الأقل:

- الزبون صاحب الطلب.
- المنتجات المطلوبة.
- خيار الكمية لكل منتج.
- السعر المعتمد وقت الطلب.
- الكمية المطلوبة من كل منتج.
- إجمالي قيمة الطلب.
- عنوان التوصيل.
- حالة الطلب.
- تاريخ الإنشاء والتحديث.

يجب حفظ السعر المعتمد وقت إنشاء الطلب، بحيث لا يؤدي تعديل سعر المنتج لاحقًا إلى تغيير قيمة الطلبات القديمة.

## دورة حياة الطلب

يجب تصميم حالات الطلب بشكل واضح، وفق التسلسل التالي:

1. **Pending**: تم إرسال الطلب وينتظر مراجعة الموظف.
2. **Approved**: تمت الموافقة على الطلب.
3. **Preparing**: يجري تجهيز الطلب.
4. **Out for Delivery**: خرج الطلب للتوصيل.
5. **Completed**: تم توصيل الطلب وإكماله.
6. **Rejected**: تم رفض الطلب.

يجب ألا يُعتبر الطلب مكتملًا بمجرد إنشائه، بل بعد إتمام عملية التوصيل.

يجب تنفيذ التحقق من انتقالات الحالات، ومنع الانتقالات غير المنطقية، مع السماح للموظفين المخولين فقط بتغيير حالة الطلب.

# 6. عناوين التوصيل المحفوظة

يكتب الزبون عنوان التوصيل كنص عند إنشاء الطلب.

يجب أن يحتفظ النظام بالعناوين التي أدخلها الزبون ليتمكن من اختيارها في الطلبات اللاحقة، مما يقلل الوقت والجهد اللازمين لإتمام الطلب.

يجب ربط العناوين بصاحبها، ومنع أي زبون من الاطلاع على عناوين زبون آخر أو استخدامها.

# 7. تسجيل الدخول والمصادقة

يجب تنفيذ المصادقة باستخدام **JWT** ومراعاة المتطلبات التالية:

- تسجيل الزبون باستخدام رقم الهاتف وكلمة المرور.
- تسجيل الدخول وإصدار **JWT** صالح للاستخدام في الطلبات المحمية.
- التحقق من هوية المستخدم وصلاحياته عبر **Middleware**.
- تطبيق Role-Based Access Control للفصل بين صلاحيات **Admin** و**Employee** و**Customer**.
- تشفير كلمات المرور باستخدام مكتبة مناسبة مثل **bcrypt** أو **bcryptjs**.
- عدم إرجاع كلمات المرور أو تخزينها بصيغة نصية واضحة.
- منع الوصول إلى البيانات أو العمليات غير المسموح بها.
- دعم جلسة دخول مستمرة لا تنتهي تلقائيًا بسبب مرور الوقت، وفق متطلبات المشروع، بحيث يستمر تسجيل الدخول حتى يسجل المستخدم الخروج.

**ملاحظة أمنية:** إذا تم تنفيذ **JWT** دون انتهاء صلاحية، فيجب توفير آلية عملية لإبطال الرموز عند تسجيل الخروج أو حظر الحساب، مع مراعاة أن **JWT** لا يمكن إبطاله تلقائيًا بمجرد حذفه من جهاز المستخدم. يجب شرح الحل المختار في المشروع.

# 8. إدارة المبيعات والإحصائيات

يجب توفير **Endpoints** تسمح لصاحب المحل بالاطلاع على إحصائيات مفيدة، مثل:

- إجمالي عدد الطلبات.
- عدد الطلبات حسب الحالة.
- إجمالي قيمة الطلبات المكتملة.
- عدد الزبائن المسجلين.
- عدد المنتجات والأقسام.
- أكثر المنتجات طلبًا، إن أمكن استخراجها من بيانات الطلبات.

يجب التمييز بين إجمالي قيمة الطلبات وإجمالي المبيعات المكتملة، وعدم احتساب الطلبات المرفوضة أو غير المكتملة ضمن المبيعات المكتملة.

لا يُطلب بناء نظام محاسبي أو إدارة مالية متكاملة.

# 9. تحليل المتطلبات وتصميم قاعدة البيانات

يجب البدء بتحليل النظام قبل كتابة الكود.

على الطالب إعداد تحليل مختصر ومنظم يتضمن:

1. المتطلبات الوظيفية.
2. المتطلبات غير الوظيفية الأساسية.
3. أنواع المستخدمين والصلاحيات.
4. الكيانات الرئيسية في قاعدة البيانات.

5. العلاقات بين الكيانات.
6. الحقول الأساسية لكل **Model**.
7. حالات الطلب والانتقالات المسموح بها.
8. آلية المصادقة والتحقق من الصلاحيات.

يجب تصميم قاعدة البيانات باستخدام **MongoDB** و**Mongoose**، وتحديد العلاقات باستخدام **ObjectId** و**References** عند الحاجة، مع تبرير الاختيارات المهمة.

يمكن أن تتضمن قاعدة البيانات كيانات مثل:

- **User**
- **Category**
- **Product**
- **Order**
- **Address**

يجوز إضافة **Models** أخرى عند وجود حاجة حقيقية إليها، دون المبالغة في تقسيم النظام.

# 10. تصميم REST API

بعد تحليل المتطلبات وتصميم قاعدة البيانات، يجب إعداد قائمة بالـ Endpoints المطلوبة قبل البدء بالتنفيذ.

يجب أن تتضمن القائمة:

- HTTP Method
- مسار Endpoint
- وصف العملية
- الصلاحية المطلوبة
- البيانات المطلوبة في Request

- شكل الاستجابة المتوقع (Response)

يجب أن تغطي الـ API عمليات التسجيل والدخول، وإدارة المستخدمين، والأقسام، والمنتجات، والعناوين، والطلبات، والإحصائيات.

يجب مراعاة تنظيم المسارات، واستخدام HTTP Status Codes المناسبة، والتحقق من المدخلات، وإرجاع رسائل خطأ واضحة.

# 11. المتطلبات التقنية والتنظيمية

يجب الالتزام بالتقنيات التالية واستخدامها ضمن الغرض المناسب:

- **Node.js**: بيئة تشغيل التطبيق.
- **JavaScript**: لغة البرمجة الأساسية.
- **Express.js**: إنشاء الخادم وتنظيم **Routes** و**Middleware**.
- **MongoDB**: قاعدة البيانات.
- **Mongoose**: تصميم **Models** والتحقق من البيانات والتعامل مع قاعدة البيانات.
- **Morgan**: تسجيل طلبات **HTTP** أثناء التطوير.
- **jsonwebtoken**: إنشاء **JWT** والتحقق منه.
- **Nodemon**: إعادة تشغيل الخادم تلقائيًا أثناء التطوير.

كما يجب:

- تنظيم المشروع في ملفات ومجلدات واضحة.
- فصل **Routes** عن **Controllers** و**Models** و**Middleware** قدر الإمكان.
- استخدام ملف `.env` للإعدادات الحساسة.
- توفير ملف `.env.example` دون أسرار حقيقية.
- استخدام معالجة مركزية للأخطاء.
- التحقق من البيانات قبل حفظها.

- منع الوصول غير المصرح به إلى بيانات المستخدمين.
- توحيد شكل استجابات النجاح والخطأ قدر الإمكان.
- توفير ملف **README** يشرح إعداد المشروع وتشغيله واستخدامه.

# 12. المطلوب تسليمه

يجب على الطالب تسليم العناصر التالية:

1. **Requirements Analysis**: تحليل المتطلبات الوظيفية وغير الوظيفية.
2. **Database Design**: تصميم **Models** والعلاقات والحقول.
3. **API Design**: قائمة منظمة بالـ **Endpoints** والصلاحيات.
4. **Backend Source Code**: المشروع كاملًا مع تنظيم واضح.
5. **README.md**: خطوات التثبيت والتشغيل والإعداد.
6. **API Testing**: إثبات اختبار العمليات الأساسية باستخدام Postman أو أداة مشابهة، مع تضمين أمثلة للنجاح والفشل والصلاحيات.
7. **Environment Example**: ملف `.env.example` يحتوي على أسماء المتغيرات المطلوبة دون قيم سرية.

# 13. معايير التقييم المقترحة

| العلامة | المعيار |
| --- | --- |
| 10 | تحليل المتطلبات وفهم المشكلة |
| 15 | تصميم قاعدة البيانات والعلاقات |

| العلامة | المعيار |
| --- | --- |
| 15 | تنظيم المشروع وجودة الكود |
| 20 | المصادقة والأمان والصلاحيات |
| 10 | إدارة الأقسام والمنتجات |
| 15 | تنفيذ الطلبات وحالاتها |
| 5 | العناوين والإحصائيات |
| 10 | الاختبار والتوثيق ومعالجة الأخطاء |
| **100** | **المجموع** |

يجب تقييم صحة الحل وتكامله، وليس عدد الملفات أو عدد الـ **Endpoints** فقط.

# 14. قواعد تنفيذ التكليف

- موعد التسليم النهائي هو 2026/10/22.
- يجب تنفيذ المشروع باستخدام التقنيات المحددة.
- يجب البدء بالتحليل والتصميم قبل التنفيذ.
- يجب أن يكون الكود من كتابة الطالب وفهمه، وأن يكون قادرًا على شرح قراراته التقنية.
- يجب اختبار العمليات الأساسية والصلاحيات والحالات الاستثنائية.
- لا يُطلب تطوير **Frontend** أو تطبيق موبايل أو رفع المشروع إلى متجر التطبيقات.

- لا يُطلب نظام دفع إلكتروني أو إدارة مالية متقدمة.
- لا يُسمح بإضافة صور للمنتجات.
- المطلوب نظام **Backend** متكامل وبسيط وقابل للتطوير، وليس نظامًا معقدًا يتجاوز أهداف التكليف.

# 15. ملاحظة ختامية للطلاب

تعامل مع هذا التكليف كما لو أنك تطور نظامًا حقيقيًا لصاحب متجر يحتاج إلى حل مشكلة عملية.

لا تبدأ بكتابة الكود مباشرة؛ افهم المشكلة، وحدد الكيانات والعلاقات، وصمم الـ API، ثم ابدأ بالتنفيذ.

الهدف ليس كتابة أكبر كمية من الكود، بل بناء **Backend** صحيح، آمن، منظم، ومتكامل.